



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026-27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Section No.: 10

Team No.: 5

Project Title: Linux Inter-Process Communication Using Pipes and FIFOs

Team Members

Roll Number	Student Name	Signature
2520080005	A.Lakshmi Sai Teja	
2520080035	B.Deepak	
2520080061	E.Pranay Reddy	

1. Abstract

Linux Inter-Process Communication Using Pipes and FIFOs is a project designed to demonstrate how different processes communicate and exchange information within a Linux operating system. Inter-Process Communication (IPC) is an important concept in operating systems that enables processes to share data, coordinate their activities, and work together efficiently. This project focuses mainly on two IPC mechanisms: Pipes and FIFOs (Named Pipes).

Pipes provide a simple communication channel between related processes, such as a parent process and its child process. One process can write data into the pipe while another process reads the data from it. The project demonstrates the creation of pipes and the use of Linux system calls such as `read()`, `write()`, and `close()` to transfer information between processes.

FIFOs, also known as Named Pipes, provide communication between independent processes. Unlike ordinary pipes, FIFOs have a name in the file system, allowing unrelated processes to access the same communication channel. The project demonstrates how FIFOs can be created and used to send and receive data between separate processes.

Overall, this project provides practical knowledge of process communication, synchronization,

data sharing, and Linux system calls. It helps in understanding how the Linux operating system manages communication between processes and how Pipes and FIFOs can be used to build efficient and reliable multi-process applications.

2. Problem Statement

In a Linux operating system, multiple processes often need to communicate and exchange data with each other. Since processes normally have separate memory spaces, direct data sharing is not possible. Therefore, an efficient Inter-Process Communication (IPC) mechanism is required.

This project aims to develop a system that demonstrates communication between processes using Pipes and FIFOs (Named Pipes). It focuses on sending and receiving data between related and independent processes using Linux system calls such as `pipe()`, `mkfifo()`, `read`, `write()`, `close()`.

The main problem is to understand how processes can transfer information safely, coordinate their execution, and communicate efficiently without directly accessing each other's memory. The project provides a practical implementation of these IPC mechanisms.

By implementing Pipes and FIFOs, the project helps demonstrate the differences between communication methods and their appropriate use in multi-process Linux applications.

3. Objectives

- To understand the concept of Inter-Process Communication (IPC) in the Linux operating system.
- To implement Pipes for communication and data exchange between related process.
- To implement FIFOs (Named Pipes) for communication between independent processes.
- To demonstrate process synchronization and data sharing using Linux IPC system calls such as
- `read()`, `write()`, and `close()`.

4. Proposed Methodology

The proposed methodology begins by creating and managing processes in the Linux operating system using system calls such as `fork()`. The project identifies the relationship between processes and establishes suitable communication channels for exchanging information. First, Pipes are implemented to enable communication between related processes. A pipe is created using the `pipe()` system call, where one process writes data using `write()` and the other process reads the data using `read()`. Proper closing of unused pipe ends is performed using `close()`. Next, FIFOs (Named Pipes) are implemented to allow communication between independent processes. The FIFO is created using `mkfifo()`, and separate processes use `read()` and `write()` to exchange data through the named communication channel.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process Management	Creation and management of processes using <code>fork()</code> .
Inter-Process Communication (IPC)	Communication between processes using Pipes and FIFOs.
Pipes	Used for communication between related processes using <code>pipe()</code> .
FIFOs (Named Pipes)	Used for communication between independent processes using <code>mkfifo()</code> .
File Descriptors	Used to identify and access communication channels.
Process Synchronization	Ensures processes communicate in the correct sequence and avoid conflicts

6. Individual Contribution

Roll Number	Student Name	Individual Responsibility
2520080005	A.Lakshmi Sai Teja	Responsible for testing the project, debugging errors, preparing documentation, screenshots, and presentation.
2520080035	B.Deepak	Pipe Communication: Responsible for implementing data communication between related processes using Pipes and Linux system calls.
2520080061	E.Pranay Reddy	Responsible for creating and implementing FIFOs (Named Pipes) for communication between independent processes.

7. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Operating system for developing and executing the IPC programs.
C / C++	Programming languages used to implement Pipes and FIFOs.
GCC Compiler	Compiles and runs the C/C++ programs.
Linux System Calls	Used for process communication through pipe(), mkfifo(), read(), write(), and close().
Terminal	Used to execute, test, and monitor the programs.

8. Expected Outcome

The project is expected to successfully demonstrate Inter-Process Communication (IPC) in Linux using Pipes and FIFOs. It will show how related processes can exchange data using Pipes and how independent processes can communicate using FIFOs.

The project will provide practical knowledge of Linux system calls, process synchronization, file descriptors, and data sharing. The programs are expected to run successfully in the Linux terminal and display reliable data transfer between processes.

9. GitHub Repository

GitHub Repository Link: [https://github.com/bdeepakg-boop/KLH-AI-DS-2026-](https://github.com/bdeepakg-boop/KLH-AI-DS-2026-TEAM5-FIFO-IPC)

TEAM5-FIFO-IPC Repository Created: ____

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: Mr. Raghupathi Manthena

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☒ Yes ☐ No

Date: _____

Faculty Signature: _____